



SAMEER RAJ

230968336

WEEK 1

Q1. Implementation of a queue in Python using two stacks

```
In [1]: class QueueUsingStacks:
    def __init__(self):
        self.stack1 = []
        self.stack2 = []

    def enqueue(self, item):
        self.stack1.append(item)
        print(f"Enqueued: {item}")

    def dequeue(self):
        if not self.stack2:
            if not self.stack1:
                print("Queue is empty")
                return None

            while self.stack1:
                self.stack2.append(self.stack1.pop())

        return self.stack2.pop()

queue = QueueUsingStacks()
queue.enqueue(45)
queue.enqueue(78)
queue.enqueue(92)
queue.enqueue(110)

print("Dequeued:", queue.dequeue())
print("Dequeued:", queue.dequeue())
```

```
Enqueued: 45
Enqueued: 78
Enqueued: 92
Enqueued: 110
Dequeued: 45
Dequeued: 78
```

Q2. Implement the following graph using python. Print the adjacency list and adjacency matrix

```
In [2]: vertices = [1, 2, 3, 4]

adjacency_list = {
    1: [(2, 1), (3, 1)],
    2: [(3, 3)],
    3: [(4, 4)],
    4: [(1, 5)]
}

print("Adjacency List:")
for vertex in adjacency_list:
    print(f"{vertex}: {adjacency_list[vertex]}")
```

Adjacency List:
1: [(2, 1), (3, 1)]
2: [(3, 3)]
3: [(4, 4)]
4: [(1, 5)]

```
In [3]: n = len(vertices)
adjacency_matrix = [[0 for _ in range(n)] for _ in range(n)]

for src in adjacency_list:
    for dest, weight in adjacency_list[src]:
        adjacency_matrix[src - 1][dest - 1] = weight

print("\nAdjacency Matrix:")
for row in adjacency_matrix:
    print(row)
```

Adjacency Matrix:
[0, 1, 1, 0]
[0, 0, 3, 0]
[0, 0, 0, 4]
[5, 0, 0, 0]

Q3. Create two list X and Y with some set of numerical values. Compute Euclidean distance for corresponding values in X and Y. Store the distance values in a separate list and sort them using Bubble sort algorithm.

In [4]:

```
import math

X = [1, 4, 6, 9]
Y = [3, 10, 2, 15]

distance = []
for i in range(len(X)):
    d = math.sqrt((Y[i] - X[i]) ** 2)
    distance.append(d)

print("Euclidean Distances:", distance)

n = len(distance)
for i in range(n):
    for j in range(0, n - i - 1):
        if distance[j] > distance[j + 1]:
            distance[j], distance[j + 1] = distance[j + 1], distance[j]

print("Sorted Distances:", distance)
```

Euclidean Distances: [2.0, 6.0, 4.0, 6.0]

Sorted Distances: [2.0, 4.0, 6.0, 6.0]

Q4. Implement the given binary search tree using Python and print the pre-order, in-order, and post-order tree traversal.

In [5]:

```
class Node:
    def __init__(self, key):
        self.key = key
        self.left = None
        self.right = None

def insert(root, key):
    if root is None:
```

```

        return Node(key)
    if key < root.key:
        root.left = insert(root.left, key)
    else:
        root.right = insert(root.right, key)
    return root

def inorder(root):
    if root:
        inorder(root.left)
        print(root.key, end=" ")
        inorder(root.right)

def preorder(root):
    if root:
        print(root.key, end=" ")
        preorder(root.left)
        preorder(root.right)

def postorder(root):
    if root:
        postorder(root.left)
        postorder(root.right)
        print(root.key, end=" ")

values = [25, 15, 50, 10, 22, 35, 70, 4, 12, 18, 24, 31, 44, 66, 90]

root = None
for v in values:
    root = insert(root, v)

print("In-order Traversal:")
inorder(root)

print("\n\nPre-order Traversal:")
preorder(root)

print("\n\nPost-order Traversal:")
postorder(root)

```

In-order Traversal:

4 10 12 15 18 22 24 25 31 35 44 50 66 70 90

Pre-order Traversal:

25 15 10 4 12 22 18 24 50 35 31 44 70 66 90

Post-order Traversal:

4 12 10 18 24 22 15 31 44 35 66 90 70 50 25

In []: